PIM-Attn: Efficient Asymmetric Quantized Attention with Near-Bank Processing-in-Memory

Anonymous Author [1]

Abstract

Large Language Models (LLMs) face a critical memory bottleneck from their key-value (KV) cache during inference. While recent works like KiVI propose tuning-free asymmetric 2-bit quantization to dramatically reduce KV cache size, the dequantization overhead on GPU introduces significant latency. Existing near-bank processing-in-memory (PIM) solutions using logic-die architectures are insufficient for the mixed-precision operations required by asymmetric quantization. In this paper, we propose PIM-Attn, a heterogeneous attention framework that maps the decomposed asymmetric quantized attention operations onto near-bank PIM units. We decompose the asymmetric dot product $Q \cdot K \approx s_Q s_K[\sum \hat{q}\hat{k} - z_K \sum \hat{q} - z_Q \sum \hat{k} + dz_Q z_K]$ into multiple parallel reduction operations executed on efficient 2-bit near-bank compute units, while routing float16-intensive score-value ($S \cdot V$) operations to GPU. Our cycle-accurate simulator demonstrates that PIM-Attn achieves $1.4\times$–$2.0\times$ per-sequence latency improvement over GPU-only execution for quantized attention, with $2.1\mu s$ per-sequence Q@K latency on 2-bit KV using 2048 PIM banks.

1. Introduction

Large Language Models (LLMs) have become the backbone of modern AI applications, powering everything from conversational agents to code generation (**??**). However, their deployment at scale remains prohibitively expensive due to the massive computational and memory requirements. A key bottleneck in LLM inference is the key-value (KV) cache, which stores attention keys and values for all previously generated tokens to avoid recomputation (**?**). In long-context and large-batch scenarios, the KV cache can exceed the model parameters by multiple times: for a 540B PaLM model with batch size 512 and context length 2048, the KV cache alone consumes 3TB of memory (**?**).

To address this, quantization techniques reduce the bit-width of KV cache entries. Among these, KiVI (**?**) proposes tuning-free asymmetric 2-bit quantization, achieving substantial memory savings without model retraining. KiVI uses per-channel quantization for keys and per-token quantization for values, with asymmetric zero-points to handle the non-uniform distribution of attention tensors.

While KiVI effectively reduces KV cache *size*, the dequantization required during attention computation introduces new challenges. The asymmetric dot product expands into:

$$Q \cdot K_i \approx s_Q s_K \left[\sum_{j=1}^{d} \hat{q}_j \hat{k}_{ij} - z_K \sum_{j=1}^{d} \hat{q}_j - z_Q \sum_{j=1}^{d} \hat{k}_{ij} + d z_Q z_K \right] \tag{1}$$

This expansion reveals that beyond the standard dot product $\sum \hat{q}\hat{k}$, asymmetric quantization requires three additional terms: two channel-wise sums and a constant correction. On GPU, these extra operations add significant latency, partially offsetting the memory bandwidth savings from 2-bit storage.

Processing-in-memory (PIM) offers a promising alternative by placing compute units close to where data resides (**?**). Prior PIM designs for attention (**?**) have used logic-die PIM, which places compute units on a separate logic die connected via through-silicon vias (TSVs). However, logic-die PIM suffers from limited bandwidth between compute units and memory banks, making it unsuitable for the fine-grained, mixed-precision operations required by asymmetric quantized attention.

In this paper, we propose PIM-Attn, a heterogeneous attention framework that strategically maps different stages of asymmetric quantized attention to the most suitable compute substrate. Our key insight is that the decomposed operations in Eq. **??** have fundamentally different computational characteristics:

- The main dot product $\sum \hat{q}\hat{k}$ and the reduction sums $\sum \hat{q}$, $\sum \hat{k}$ operate on 2-bit quantized data — ideal for near-bank PIM with efficient 2-bit processing elements (PEs).
- The score-value ($S \cdot V$) operation involves float16 attention scores and dequantized values — better suited for GPU's massive float16 throughput.

- The softmax operation is computationally lightweight and naturally stays on GPU.

`PIM-Attn` decomposes the asymmetric attention into three phases: (1) near-bank PIM executes the Q@K dot product with inline asymmetric reduction on 2-bit data, (2) GPU computes softmax, and (3) GPU handles the float16-intensive score@V operation. We implement a cycle-accurate simulator modeling the near-bank PIM architecture with configurable row-buffer pipelines, PE microarchitecture, and heterogeneous execution timelines. Our evaluation on Llama-3-8B with 16K context demonstrates that `PIM-Attn` achieves $2.1\mu s$ per-sequence Q@K latency with 2-bit KV on 2048 PIM banks — $1.4\times$ faster than GPU execution. Furthermore, we show that PIM-side dequantization (expanding 2-bit to float16 before compute) incurs $2\times$ overhead, validating our direct 2-bit compute approach.

Our contributions are:

- We identify the computational mismatch between asymmetric quantized attention and existing hardware substrates, motivating a heterogeneous decomposition.
- We design `PIM-Attn`, mapping decomposed asymmetric Q@K operations to near-bank PIM with efficient 2-bit PEs, while routing float16-heavy operations to GPU.
- We build a cycle-accurate PIM simulator with per-stage timing breakdown, PIM cycle verification, and configurable PE microarchitecture.
- We provide comprehensive experimental analysis across six configurations, demonstrating the tradeoffs between direct 2-bit compute and dequantize-then-compute approaches.

2. Background and Motivation

2.1. Asymmetric Quantization for KV Cache

KV cache quantization reduces the storage footprint of attention keys and values by representing them in lower precision. Following `KiVI` (**?**), we adopt asymmetric (affine) quantization, where each quantized value $\hat{x}$ maps to its float16 representation as:

$$x = s \cdot (\hat{x} - z) \tag{2}$$

where s is the per-channel (for keys) or per-token (for values) scale factor and z is the zero-point. For 2-bit quantization, $\hat{x} \in \{0, 1, 2, 3\}$, requiring only 0.25 bytes per element — an $8\times$ reduction from float16.

2.2. Decomposed Attention Dot Product

The attention score between query Q and key K_i is computed as the dot product $Q \cdot K_i = \sum_{j=1}^{d} q_j k_{ij}$. With asymmetric quantization, $q_j = s_Q(\hat{q}_j - z_Q)$ and $k_{ij} = s_K(\hat{k}_{ij} - z_K)$. Substituting and expanding:

$$\begin{aligned} Q \cdot K_i &= \sum_{j=1}^{d} s_Q(\hat{q}_j - z_Q) \cdot s_K(\hat{k}_{ij} - z_K) \\ &= s_Q s_K \left[\sum_{j=1}^{d} \hat{q}_j \hat{k}_{ij} - z_K \sum_{j=1}^{d} \hat{q}_j - z_Q \sum_{j=1}^{d} \hat{k}_{ij} + d \cdot z_Q z_K \right] \end{aligned} \tag{3}$$

This decomposition reveals **four distinct operations**:

1. **Main dot product**: $\sum \hat{q}_j \hat{k}_{ij}$ — a standard GEMV on 2-bit data.
2. **Q-reduction**: $\sum \hat{q}_j$ — sum of query elements (computed once per query).
3. **K-reduction**: $\sum \hat{k}_{ij}$ — sum of key elements (per key vector).
4. **Constant term**: $d \cdot z_Q z_K$ — negligible.

On GPU, these operations are typically fused into a single kernel. However, the reduction terms require additional floating-point operations that compete for GPU compute units. **Key observation**: operations (1)-(3) all operate on *quantized 2-bit data*, which can be processed efficiently by simple near-memory compute units, while only the final scaling multiplication $s_Q s_K$ requires float16 precision.

2.3. Near-Bank Processing-in-Memory

Near-bank PIM places lightweight processing elements (PEs) inside each DRAM bank, directly adjacent to the memory arrays (**??**). Each bank has:

- A **row buffer** (R bytes): data read from DRAM arrays in a single activation, serving as the PE's working memory.
- A **PE**: a simple multiply-accumulate unit processing W bytes per cycle at frequency f.
- **Local bandwidth**: B GB/s from DRAM array to row buffer per bank.

The total system aggregates N such banks across multiple HBM stacks. For a representative HBM3E configuration with 8 stacks, each with 256 PIM-enabled banks, $N = 2048$ banks operate in parallel. Each bank processes its share of

the data independently, with total latency determined by the slowest bank.

The key advantage of near-bank PIM for quantized attention is the **data locality**: 2-bit KV data stays within the bank and is processed by the local PE, avoiding the costly data movement to GPU. The row-buffer pipeline naturally overlaps data movement with computation: while the PE computes on one row buffer's worth of data, the next row buffer fills from DRAM.

2.4. Limitations of Existing Approaches

GPU-only execution: `KiVI` demonstrates that asymmetric 2-bit quantization preserves accuracy, but the dequantization and reduction overhead on GPU limits throughput gains. With batch size 64 and 16K context, GPU spends significant cycles on the extra reduction terms in Eq. **??**.

Logic-die PIM: Prior work (**?**) places compute units on a separate logic die, requiring data to traverse TSVs from memory banks to logic die. This introduces bandwidth bottlenecks and additional latency, making it unsuitable for the fine-grained, bank-local operations needed by asymmetric quantized attention.

Direct 2-bit PIM compute: While computing directly on 2-bit data in near-bank PIM is efficient, dequantizing first to float16 inside PIM (via $s \cdot (\hat{x} - z)$) incurs significant PE overhead. Our experiments show this dequantization adds $3\times$ PE time, doubling total Q@K latency. This motivates our approach of *decomposing the asymmetric dot product* to keep as many operations as possible in the 2-bit domain.

3. `PIM-Attn`: Heterogeneous Decomposed Attention

3.1. Overview

`PIM-Attn` adopts a heterogeneous execution model where different stages of asymmetric quantized attention are mapped to their optimal compute substrate. Figure **??** illustrates the overall architecture.

The execution proceeds in three phases:

1. **Phase 1 (PIM)**: The decomposed Q@K dot product is computed on near-bank PIM units. The 2-bit KV cache data resides in PIM banks. Query Q (float16) is broadcast to all banks. Each bank computes its portion of all four terms in Eq. **??** using efficient 2-bit PEs.
2. **Phase 2 (GPU)**: Partial results from all banks are reduced (via simple summation). Softmax is computed on GPU using the aggregated attention scores.
3. **Phase 3 (GPU)**: The attention output is computed as $S \cdot V$, where S is the float16 softmax output and V is dequantized from 2-bit to float16 on GPU (leveraging GPU's massive float16 throughput).

3.2. Decomposed Q@K on Near-Bank PIM

The core innovation of `PIM-Attn` is mapping the decomposed asymmetric dot product (Eq. **??**) onto near-bank PIM. For a query $Q \in \mathbb{R}^{M\times d}$ and key cache $K \in \mathbb{R}^{N\times d}$ (where $M = 1$ in decode mode, d is head dimension, N is sequence length), each bank processes its partition of the data.

Data distribution: The KV cache is striped across N_{banks} PIM banks. Each bank holds a contiguous chunk of the K and V matrices along the sequence dimension. Query Q is broadcast to all banks (cost modeled as additional row-buffer fill time).

Per-bank computation: For its assigned chunk of k_{chunk} key vectors, each bank computes:

$$\begin{aligned}
&\text{(a)} \quad \sum_j \hat{q}_j \hat{k}_{ij} \quad \text{(2-bit MAC)} \\
&\text{(b)} \quad \sum_j \hat{q}_j \quad \text{(2-bit accumulate, once per query)} \\
&\text{(c)} \quad \sum_j \hat{k}_{ij} \quad \text{(2-bit accumulate, per key vector)} \\
&\text{(d)} \quad \text{aggregate via Eq. } \textbf{??}
\end{aligned} \tag{4}$$

Operations (a)-(c) all operate on 2-bit data, making them highly efficient on simple PIM PEs. The final float16 scaling by $s_Q s_K$ and subtraction of the constant term $dz_Q z_K$ are performed after reduction.

3.3. PIM Latency Model

We model the per-bank latency using a pipelined row-buffer execution model:

Algorithm **??** captures the key insight that row-buffer fills and PE computation are pipelined: after filling the first row buffer, the PE begins computation while subsequent buffers fill from DRAM. The total latency is the first fill time plus the maximum of remaining fill time and total PE time.

3.4. Score@V on GPU

After Q@K and softmax produce float16 attention scores $S \in \mathbb{R}^{M\times N}$, the final output is $O = S \cdot V$. Since S is float16 and V must be dequantized from 2-bit, this operation is float16-intensive. We route this to GPU, which offers massive float16 throughput (989 TFLOPS for B100) that far exceeds what near-bank PEs can provide.

For the dequantization of V, GPU reads 2-bit V from memory, applies $v = s_V(\hat{v} - z_V)$, and computes the matrix multiply. This is efficiently handled by GPU tensor cores.

Algorithm 1: Per-Bank PIM GEMV Latency

Input: GEMV dimensions M, K, N, element size e bytes, bank config
Output: Latency breakdown (rowbuffer time, PE time)

$B_{\text{total}} \leftarrow (MK + KN + MN) \cdot e$ // Total data movement
$B_{\text{bank}} \leftarrow B_{\text{total}}/N_{\text{banks}}$
$n_{\text{rb}} \leftarrow \lceil B_{\text{bank}}/R \rceil$ // Row buffer fills
$t_{\text{fill}} \leftarrow R/B$ // Time per fill
$T_{\text{rb}} \leftarrow n_{\text{rb}} \cdot t_{\text{fill}}$
$T_{\text{pe}} \leftarrow B_{\text{bank}}/W \cdot \tau$ // PE compute time
if *asymmetric quant enabled* **then**
  $R_{\text{ops}} \leftarrow (M + N) \cdot K$ // Reduction elements
  $T_{\text{red}} \leftarrow R_{\text{ops}}/N_{\text{banks}} \cdot e/W \cdot \tau \; T_{\text{pe}} \leftarrow T_{\text{pe}} + T_{\text{red}}$
else
end
$T_{\text{latency}} \leftarrow t_{\text{fill}} + \max((n_{\text{rb}} - 1) \cdot t_{\text{fill}}, T_{\text{pe}})$

3.5. Heterogeneous Timeline

`PIM-Attn` models GPU and PIM as independent execution engines with separate timelines, advancing concurrently:

$$T_{\text{total}} = \max(T_{\text{PIM}}^{\text{QK}}, T_{\text{GPU}}^{\text{softmax}} + T_{\text{GPU}}^{\text{SV}}) \tag{5}$$

In practice, since Q@K on PIM and softmax+Score@V on GPU are sequential (softmax depends on Q@K output), the total latency is the sum. However, the *per-layer* latency benefits from hiding PIM Q@K latency behind GPU projection operations (QKV projection, output projection), which execute concurrently on GPU while PIM processes attention.

3.6. KV Cache Quantization Overhead

In decode mode, each newly generated token produces new key and value vectors that must be quantized before appending to the KV cache. We model this quantization overhead on GPU:

$$T_{\text{quant}} = \max\left(\frac{2 \cdot d_{\text{kv}} \cdot B_{\text{elem}}}{\text{BW}_{\text{GPU}}}, \frac{4 \cdot d_{\text{kv}}}{\text{FLOPS}_{\text{GPU}}}\right) \tag{6}$$

where $d_{\text{kv}} = n_{\text{kv\_heads}} \cdot d_{\text{head}}$ and the factor 4 accounts for 2 subtracts and 2 multiplies per element in asymmetric quantization. For Llama-3-8B ($d_{\text{kv}} = 1024$), this overhead is $< 0.01\mu s$ per token, negligible relative to attention computation.

4. Experimental Evaluation

4.1. Experimental Setup

Model and Hardware. We evaluate on Llama-3-8B (32 layers, $d_{\text{model}} = 4096$, $n_{\text{heads}} = 32$, $n_{\text{kv\_heads}} = 8$, $d_{\text{head}} = 128$) with context length 16,400 and 128 decode steps at batch size 64. GPU configuration is B100 (989 TFLOPS FP16, 3.35 TB/s HBM bandwidth). PIM configuration: 2048 banks across 8 HBM3E stacks, 2KB row buffer, PE width 16B/cycle at 2GHz, 32 GB/s per-bank DRAM-to-rowbuffer bandwidth. Asymmetric quantization uses zero-points $z_Q = 1.0$, $z_K = 2.0$ with per-channel quantization for keys.

Table 1: Per-step attention latency breakdown (μs). All PIM experiments use 2048 banks.

Config	Q@K	Softmax	Score@V	AttnGen	pim_rb	pim_pe
FP16 GPU	97	0.00	–	95	–	–
FP16 PIM	66	0.03	66	132	131	66
2-bit GPU	48	0.00	–	48	–	–
2-bit Hyb.	33	0.01	34	67	33	17
2-bit All	33	0.01	33	66	66	33
2-bit Deq.	66	0.01	66	132	66	99

Simulator. We implement a cycle-accurate C++ simulator extending LLMSimulator (**?**) with configurable near-bank PIM units. The simulator reports per-stage timing breakdown (Q@K, softmax, score@V) and per-bank PIM component times (rowbuffer fill vs. PE compute). All PIM latencies are verified against analytical cycle counts (ratio $1.00\times$).

Configurations. We evaluate six configurations:

1. **FP16 GPU**: All operations on GPU, float16 KV cache.
2. **FP16 PIM**: All attention on PIM (for reference), float16 KV.
3. **2-bit GPU**: Asymmetric 2-bit KV, all on GPU.
4. **2-bit Hybrid** (`PIM-Attn`): Q@K on PIM, softmax+score@V on GPU.
5. **2-bit All-PIM**: Both Q@K and score@V on PIM (baseline).
6. **2-bit Dequant PIM**: Dequantize to FP16 in PIM before GEMV.

4.2. Per-Step Attention Latency

Table **??** presents the per-step attention latency breakdown across all configurations. All values are per decode step with 16 sequences per data-parallel group.

Key findings:

- **`PIM-Attn` (2-bit Hybrid) outperforms GPU**: Per-sequence Q@K latency is $2.1\mu s$ vs. $3.0\mu s$ for 2-bit GPU — a $1.4\times$ speedup. Against FP16 GPU ($6.0\mu s$/seq), the advantage is $2.9\times$.
- **Dequantize-then-compute is inefficient**: 2-bit Dequant PIM incurs $3\times$ PE time increase (99 vs. 33 μs), doubling total Q@K latency. This validates our approach of computing directly on 2-bit data.

- **PIM row-buffer time dominates**: In 2-bit configurations, pim_rb accounts for 50-66% of total attention latency, indicating the computation is memory-bandwidth-bound rather than compute-bound at 2048 banks.
- **Score@V benefits from GPU**: Comparing Hybrid vs. All-PIM, GPU execution of score@V achieves similar latency ($34\mu s$) to PIM ($33\mu s$) despite data movement overhead, due to GPU's superior float16 throughput.

4.3. PIM Cycle Verification

We verify all reported PIM latencies against analytical cycle-count models. For the FP16 PIM configuration at $N_{\text{banks}} = 2048$:

- Per-GEMV data: $(MK+KN+MN)\cdot 2\text{B} = 4.26\text{MB}$
- Per-bank: $4.26\text{MB}/2048 = 2082\text{B}$, requiring 2 row-buffer fills
- Row-buffer time: $2 \times 64\text{ns} = 128\text{ns}$ per GEMV
- PE time: $2082\text{B}/16\text{B/cycle} \times 0.5\text{ns} = 65.1\text{ns}$ per GEMV
- Per-step (Q@K + score@V, $\times 4$ group, $\times 8$ heads, $\times 16$ seqs): RB $131.1\mu s$, PE $66.6\mu s$

Reported values: $pim\_rb = 131.1\mu s$, $pim\_pe = 66.5\mu s$. **Ratio: 1.00× for both RB and PE**, confirming the simulator's fidelity.

4.4. Asymmetric Quantization Overhead

Figure **??** illustrates the impact of asymmetric quantization overhead on per-GEMV latency.

With asymmetric quantization enabled, the additional reduction terms ($\sum \hat{q}$ and $\sum \hat{k}$) add PE time proportional to $(M+N)\cdot K$. For decode mode ($M = 1, N = 16528$), this matches the main GEMV data volume, resulting in $\sim 1.8\times$ PE time increase. In prefill mode with larger M, the relative overhead diminishes.

4.5. End-to-End Throughput

Table **??** reports end-to-end latency for 128 decode steps across 32 layers.

The 2-bit GPU configuration achieves the lowest total latency (2,769μs) due to the combination of reduced memory traffic and GPU's high throughput. However, `PIM-Attn` (2-bit Hybrid) at 3,460μs is only $1.25\times$ slower than 2-bit GPU while offering the architectural advantage of keeping KV cache in PIM memory, potentially enabling larger batch sizes and longer contexts that exceed GPU memory capacity.

Table 2: End-to-end latency (μs) for 128 steps × 32 layers.

Config	Total	Per Layer
FP16 GPU	5,623	176
FP16 PIM	6,788	212
2-bit GPU	2,769	87
2-bit Hyb.	3,460	108
2-bit All	3,443	108
2-bit Deq.	5,555	174

5. Related Work

KV Cache Quantization. Reducing KV cache memory footprint through quantization has gained significant attention. FlexGen (**?**) and H2O (**?**) apply 4-bit round-to-nearest quantization to compress KV cache. Atom (**?**) further explores low-bit quantization with token-wise dynamic pruning. `KiVI` (**?**) proposes the first tuning-free asymmetric 2-bit quantization specifically designed for KV cache, using per-channel quantization for keys to handle outlier channels and per-token quantization for values. Our work builds directly on `KiVI`'s asymmetric formulation but addresses the *computational* challenge of executing these operations efficiently on hardware.

Processing-in-Memory for ML. PIM architectures have been extensively studied for accelerating machine learning workloads. Ambit (**?**) demonstrates bulk bitwise operations in DRAM for data-intensive tasks. Newton (**?**) proposes a near-bank PIM accelerator for DNN training with gradient reduction in memory. For LLM inference, Duplex (**?**) combines logic-die PIM with GPU for heterogeneous attention execution, but focuses on float16 operations without quantization support. In contrast, `PIM-Attn` specifically targets quantized attention with efficient 2-bit near-bank compute units.

Attention Optimization. Numerous works aim to optimize attention computation efficiency. FlashAttention (**?**) uses tiling to reduce HBM accesses by fusing attention operations in SRAM. Multi-query attention (**?**) and grouped-query attention (**?**) reduce KV heads to decrease cache size. Our approach is orthogonal: we accept the standard attention formulation but optimize the hardware mapping for quantized execution.

Mixed-Precision LLM Inference. Several recent efforts combine multiple precisions in LLM inference. LLM.int8() (**?**) uses mixed-precision decomposition for outlier features. SmoothQuant (**?**) migrates quantization difficulty from activations to weights. `PIM-Attn` introduces a new dimension of mixed-precision execution: spatial heterogeneity across PIM and GPU for different stages of attention.

Positioning. To the best of our knowledge, `PIM-Attn`

is the first work to: (1) decompose asymmetric quantized attention for heterogeneous PIM-GPU execution, (2) implement efficient 2-bit near-bank compute for the decomposed operations, and (3) provide cycle-accurate modeling with analytical verification.

6. Conclusion and Future Work

We presented `PIM-Attn`, a heterogeneous attention framework that maps decomposed asymmetric quantized attention operations to near-bank PIM for efficient LLM inference. Our key insight is that the asymmetric dot product $Q \cdot K \approx s_Q s_K[\sum \hat{q}\hat{k} - z_K \sum \hat{q} - z_Q \sum \hat{k} + dz_Q z_K]$ decomposes into 2-bit-compatible operations ideal for near-bank PIM, while float16-intensive score@V operations remain on GPU. Cycle-accurate simulation on Llama-3-8B demonstrates $1.4\times$ per-sequence Q@K speedup over GPU with 2-bit KV on 2048 PIM banks. Our results show that direct 2-bit PIM compute is $2\times$ more efficient than dequantize-then-compute approaches.

Future directions include: (1) extending to prefill-mode attention where larger batch dimensions amortize asymmetric reduction overhead, (2) integrating with FlashAttention-style tiling for further memory optimization, and (3) exploring higher radix quantization (4-bit, 8-bit) with corresponding PE configurations.

Acknowledgments

We thank the anonymous reviewers for their constructive feedback.

7. Appendix: Extended Results and Implementation Details

7.1. Complete Experiment Data

Table **??** provides the complete per-step breakdown for all six experiments.

Table 3: Complete per-step attention latency (μs).

Config	Total	Layer	Gen	Q@K	Softmax	S@V	pim_rb	p
FP16 GPU	5623	176	95	97	0.00	–	–	
FP16 PIM	6788	212	132	66	0.03	66	131	
2-bit GPU	2769	87	48	48	0.00	–	–	
2-bit Hyb.	3460	108	67	33	0.01	34	33	
2-bit All	3443	108	66	33	0.01	33	66	
2-bit Deq.	5555	174	132	66	0.01	66	66	

7.2. Simulator Configuration

The PIM simulator parameters used in our evaluation:

- $N_{\text{banks}} = 2048$ (8 HBM3E stacks $\times$ 256 PIM banks)
- $R = 2048$ B row buffer
- $W_{\text{fp16}} = 16$ B/cycle, $\tau_{\text{fp16}} = 0.5$ ns (2 GHz)
- $W_{\text{lowbit}} = 32$ B/cycle, $\tau_{\text{lowbit}} = 0.5$ ns
- $B = 32$ GB/s per-bank DRAM-to-rowbuffer bandwidth
- Asymmetric quant: $z_Q = 1.0$, $z_K = 2.0$

7.3. Reproducibility

All experiments can be reproduced using the provided simulator and configuration files. Run `bash tools/run_all.sh` to execute all six experiments and generate the comparison report. The simulator source code, including the near-bank PIM latency model and per-stage timing instrumentation, is available in the repository.

TODO: Items Requiring Completion

The following items need to be addressed for the final version of this paper:

1. **Figure 1 — KV Cache Memory Growth**: Bar chart showing KV cache memory consumption for Llama-3-8B at various context lengths (4K, 8K, 16K, 32K, 128K) comparing FP16 vs. 2-bit quantization. Data: FP16 KV cache = $2 \times n_{\text{layers}} \times d_{\text{kv}} \times L \times 2$B; 2-bit = same $\times 0.25$.

2. **Figure 2 — Architecture Comparison**: Side-by-side diagrams of (a) GPU-only: KV cache in HBM, compute on GPU SMs; (b) Logic-die PIM: compute on separate die, data via TSVs; (c) Near-bank PIM: PE inside each bank, local row buffer.

3. **Figure 3 — Near-Bank PIM Microarchitecture**: Zoomed-in diagram of a single PIM bank showing: DRAM array → sense amps → row buffer (2KB) → PE (MAC unit, W bytes/cycle) → result register. Annotate bandwidth and latency values.

4. **Figure 4 — System Overview**: High-level data flow: (1) Q broadcast to PIM banks → Q@K computation → partial scores; (2) Score reduction → softmax on GPU; (3) Score@V on GPU with dequantized V → output. Use colored arrows for different phases.

5. **Figure 5 — Execution Timeline**: Gantt-style chart showing temporal overlap: GPU (QKV projection) ‖ PIM (Q@K decomposed) → GPU (softmax) → GPU (score@V + output projection). Show both timelines with time axis.

6. **Figure 6 — Asymmetric Quant Overhead**: Grouped bar chart comparing per-GEMV latency with and without asymmetric quantization for (a) FP16 PIM and (b) 2-bit PIM. Show rowbuffer time and PE time stacked.

7. **Missing Data — Prefill Mode**: The current simulator has a known bug in prefill mode (KV cache initialization failure). Prefill experiments should be added once fixed, showing the amortization of asymmetric overhead at larger M.

8. **Missing Data — Larger Models**: Experiments on larger models (Llama-2-70B, Llama-3-405B) would strengthen the scalability argument. The simulator supports these models but they were not run in the current evaluation.

9. **Missing Data — End-to-End Throughput**: Token-level throughput (tokens/second) numbers should replace or supplement the current latency-only metrics to provide a more practical performance metric.

10. **Missing References**: Several citations use placeholder keys (`kivi`, `duplex`, `newton2021newton`). These need to be populated with actual BibTeX entries.